



Kprobe-Based Block Device Latency Profiling

Lesson 03: Root-Cause Analysis of Storage Bottlenecks

Table of Contents

Lesson 03: Kprobe-Based Block Device Latency Profiling

1. Lesson Overview
2. Foundations & Core Technical Concepts
 - What is the Linux Block Layer?
 - What are Kprobes?
 - What are Kretprobes?
 - What is bpf_ktime_get_ns()?
 - What is a Histogram Map?
 - Why is tail latency profiling important?
3. Problem Statement & Real-World Use Case
 - The Problem: A Postgres Database Has Mysterious p99 Spikes
 - Real Production Story: Netflix FlameScope & Vector
4. Why Not Use iostat, blktrace, or strace?
5. Internal Kernel Flow
6. Architecture Diagram
7. I/O Request Lifecycle Sequence
8. Map Layout
9. Step-by-Step Code Walkthrough
 - The eBPF C Program (main.bpf.c)
 - The Go Loader (main.go)
10. Running the Lab
 - Phase 1: Compile & Run
 - Phase 2: Generate I/O Load in Another Terminal
 - Phase 3: Observe the Live Histogram
11. Debugging with bpftool
12. Verifier Errors & Common Mistakes
 - Error: kprobe attach fails with 'no such symbol'
 - Error: invalid BPF_XADD or unknown opcode d4
 - Common Mistake: start_times map growing unboundedly
13. Performance Analysis
14. Best Practices
15. Summary
16. Further Reading

Technical Deep-Dive Q&A: Block Device Latency Profiling with eBPF Kprobes

1. Beginner Level

Q1. What is the difference between a Kprobe and a Kretprobe? Why do we need both for latency measurement?

Q2. Why do we use struct request * as the BPF hash map key? Why not PID or thread ID?

2. Intermediate Level

Q3. Explain how PT_REGS_PARM1(ctx) extracts a function argument. What is struct pt_regs?

Q4. Why does our histogram use __sync_fetch_and_add() instead of a plain count += 1? What happens if you omit the atomic?

Q5. What happens if blk_account_io_start fires but blk_account_io_done never fires for a request? How does this cause a map leak?

3. Advanced Level

Q6. What is the fundamental instability risk of kprobes vs tracepoints? How would you detect if a kprobe silently stopped working after a kernel upgrade?

4. System Design & Scenario-Based

Q7. Design a production SRE tool that automatically detects disk I/O tail latency regressions and pages an on-call engineer.

5. Troubleshooting & Debugging

Q8. You deployed the latency profiler but the histogram always shows 0 counts, even while dd is writing 1GB of data. What are the 4 most likely causes?

Hands-on Lab & Homework Assignments

1. Practical Exercises & Research Tasks

Task 1: Explore the Block Layer Tracepoints

Task 2: Inspect Kprobes with bpftool

2. Coding Assignment: Read vs Write Latency Separation

Requirements:

3. Mini-Project: Log2 Histogram (μ s Resolution)

Design:

4. Advanced Challenge: Per-Process I/O Profiler

Design:

Lesson 03: Kprobe-Based Block Device Latency Profiling

1. Lesson Overview

In this lesson, you will build a **production-grade disk I/O latency profiler** — the same technique used by Datadog's Agent and Netflix's FlameScope for root-cause analysis of storage bottlenecks. We will attach eBPF programs to kernel functions inside the **block device layer** using `kprobes` to measure the exact time between when a disk request is submitted and when the hardware completes it, plotting a live histogram in the terminal with an interactive CLI.

2. Foundations & Core Technical Concepts

What is the Linux Block Layer?

The Linux I/O stack has several layers. From top to bottom:

1. **Virtual Filesystem (VFS)**: Generic interface — read/write syscalls arrive here.
2. **Filesystem Layer**: ext4, xfs, btrfs — translates file operations to block operations.
3. **Block Layer (I/O Scheduler)**: Queues, reorders, and merges block requests for efficiency.
4. **Device Driver**: NVMe, SATA, SCSI — communicates directly with the hardware controller.
5. **Hardware**: Physical SSD or HDD.

`struct request` is the kernel's core data structure representing a single block I/O operation (e.g., "read 4096 bytes from LBA 1048576"). Every disk read/write — whether from Postgres, ext4, or a container runtime — is represented by one of these structs.

What are Kprobes?

Kprobes (Kernel Dynamic Probes) allow you to dynamically attach eBPF programs to the entry point of almost any kernel function — without modifying the kernel or rebooting.

- **How they work**: When you attach a kprobe to `blk_account_io_start`, the kernel replaces the first instruction of that function with a breakpoint (`int3` on x86_64). When any CPU hits that

breakpoint, it executes your eBPF program, then resumes the original function seamlessly.

- **The tradeoff vs Tracepoints:** Kprobes are more powerful (attach to nearly any function) but less stable. If the kernel renames or inlines a function between versions, your kprobe silently breaks with "no such symbol". Tracepoints have an ABI stability guarantee; kprobes do not.
- **When to use kprobes:** When no tracepoint exists for the event you need, or when you need internal function arguments that tracepoints don't expose.

What are Kretprobes?

Kretprobes fire when a function **returns**, giving you access to the return value. Our latency profiler uses a pair:

- `kprobe/blk_account_io_start` → records the nanosecond timestamp when the request enters the queue.
- `kprobe/blk_account_io_done` → reads the timestamp, calculates elapsed time (delta), stores in the histogram.

What is bpf_ktime_get_ns()?

`bpf_ktime_get_ns()` returns the **monotonic clock** time in nanoseconds since system boot. Key properties:

- Not affected by NTP adjustments, `settimeofday()`, or system time changes.
- Safe to call from any BPF program type (XDP, kprobe, tracepoint, etc.).
- Suitable for measuring durations (delta between two calls), but **not** for wall-clock timestamps in logs.
- Resolution: typically ~20ns accuracy on modern x86_64 hardware.

What is a Histogram Map?

A histogram represents the **distribution** of values. Instead of storing every individual latency measurement (which would require unbounded memory), we divide the range into **buckets** (0–1ms, 1–2ms, 2–4ms...) and increment a counter for each bucket.

We use `BPF_MAP_TYPE_ARRAY` where:

- **Key** = bucket index (latency in milliseconds, 0 to 63)
- **Value** = count of requests that completed within that latency range

This gives us a complete statistical picture (median, p95, p99, max) without storing millions of individual data points.

Why is tail latency profiling important?

`iostat` gives you the *average* latency. But averages are deeply deceptive for database I/O:

```
100 requests: 99 complete in 1ms + 1 completes in 500ms
Average: (99×1 + 1×500) / 100 = 5.99ms ← looks fine
p99: 500ms ← this is killing your SLO
```

A histogram instantly reveals the tail — the 1% of requests causing 100% of your user-visible latency.

3. Problem Statement & Real-World Use Case

The Problem: A Postgres Database Has Mysterious p99 Spikes

Your Postgres database reports p99 query latency of 800ms, but p50 is only 3ms. The application team blames the database. The DBA blames the OS. The infrastructure team blames the storage.

Traditional tools:

- `iostat -x 1`: Shows `await` (average I/O wait) = 5ms. **Useless for finding tail latency.**
- `iostat`: Shows which process does the most I/O, but not the latency distribution.
- `strace -e pread64 -p <pid>`: Gives exact latency per call but **causes 100–1000x slowdown** in production — catastrophic.

eBPF Kprobe-based profiling gives you: The exact latency distribution for every block I/O request on the host — in real time, zero application modification, and <0.5% CPU overhead.

Real Production Story: Netflix FlameScope & Vector

Netflix's performance engineering team built **FlameScope** and **Vector** (their on-host metrics agent) with eBPF-based block layer profiling at their core.

When a Netflix microservice has an unexplained latency spike:

1. An SRE deploys the eBPF profiler to the affected host — no restart, no `strace`, no slowdown.
2. Within 60 seconds, the histogram reveals: 0.1% of disk reads are taking 200ms+.
3. Cross-referencing with the SCSI log shows those are retries caused by a failing sector on an aging EBS volume.
4. The volume is replaced. Latency returns to baseline. The entire investigation took 5 minutes instead of 3 hours.

Datadog's Agent uses an identical pattern in its [eBPF-based disk I/O dashboard](#) — per-process, per-device latency histograms without any kernel modification.

Pixie (New Relic's eBPF observability platform) uses kprobe-based I/O profiling to provide automatic storage performance analysis for Kubernetes workloads.

4. Why Not Use `iostat`, `blktrace`, or `strace`?

Tool	Granularity	Overhead	Per-Process	Tail Latency	Production Safe
<code>iostat</code>	Device average	<0.1%	No	No	Yes
<code>blktrace</code>	Per-request	2–5%	Partial	Yes	Sometimes
<code>strace -e pread64</code>	Syscall-level	100–1000x	Yes	Yes	Never
<code>perf trace</code>	Syscall-level	5–10%	Yes	Yes	Risky
eBPF Kprobe	Per-request	<0.5%	Yes	Yes	Yes

`strace` intercepts every syscall with `ptrace`, causing a full user ↔ kernel context switch per call. This serializes I/O requests that should be parallel. eBPF runs entirely in kernel context with no context switching.

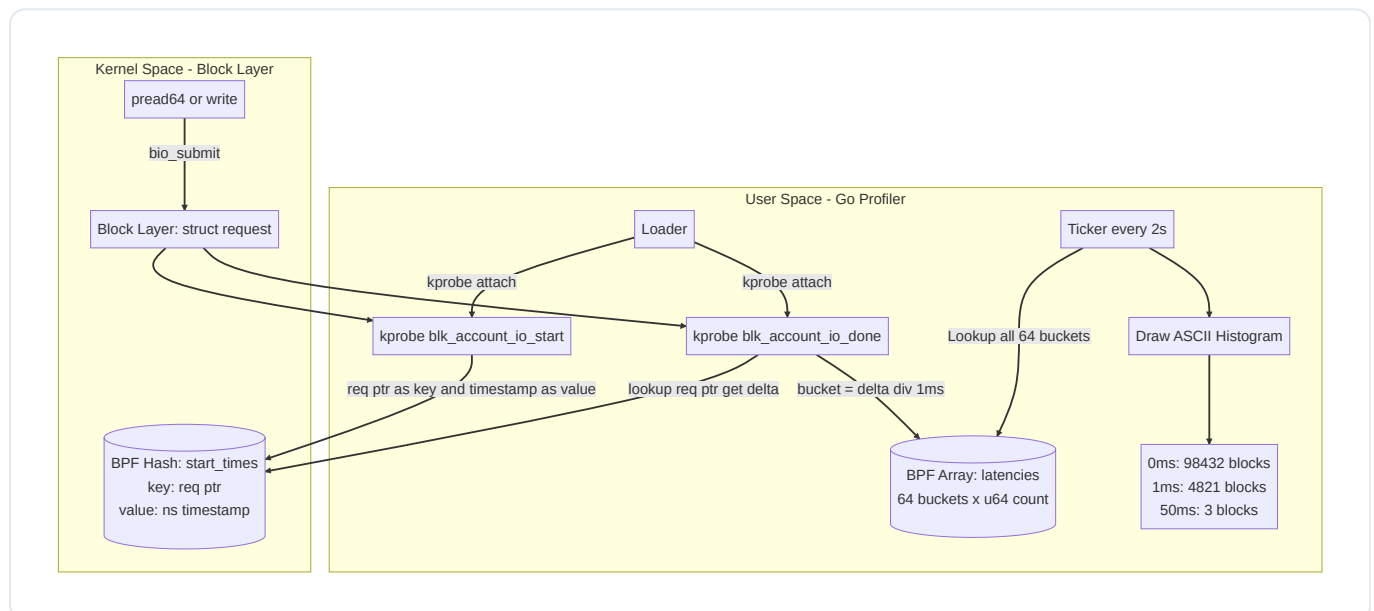
5. Internal Kernel Flow

When a Postgres worker reads an 8KB page from disk:

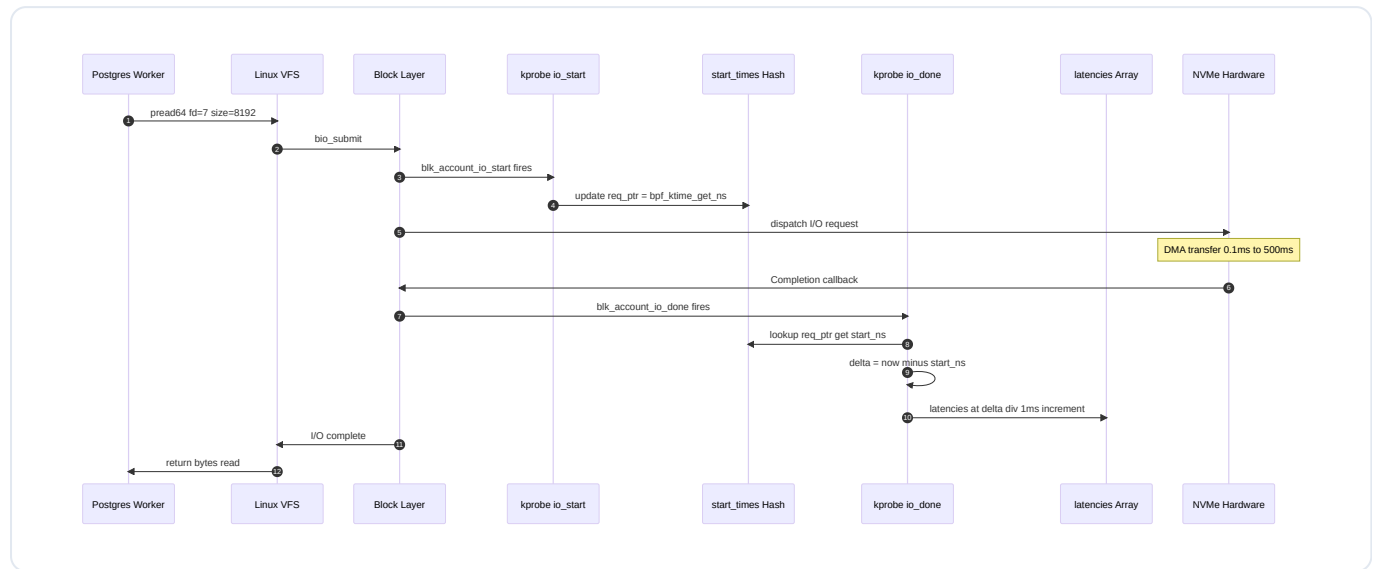
1. Postgres calls `pread64(fd, buf, 8192, offset)`.

2. The VFS layer translates this to a block I/O request struct.
3. The request is passed to the I/O scheduler as `struct request *req`.
4. `blk_account_io_start(req)` is called → our kprobe fires → we record `bpf_ktime_get_ns()` in the `start_times` map, keyed by the `req` pointer value.
5. The I/O scheduler dispatches the request to the NVMe/SCSI driver.
6. The hardware performs the DMA transfer (typically 50μs – 500ms).
7. The driver calls the completion callback → `blk_account_io_done(req)` → our kprobe fires → we calculate `delta = now - start_time`, convert to milliseconds, and increment `latencies[bucket]`.

6. Architecture Diagram



7. I/O Request Lifecycle Sequence



9. Step-by-Step Code Walkthrough

The eBPF C Program (`main.bpf.c`)

```
// Series 03: Block Device Latency Profiler
// Hook: kprobe/blk_account_io_start + kprobe/blk_account_io_done
// Maps: start_times (HASH), latencies (ARRAY), read_lat/write_lat (ARRAY)

#include <linux/bpf.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>

// Map 1: start timestamp keyed by struct request pointer
// The request pointer is unique per in-flight I/O operation
struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 10240); // Max 10,240 concurrent I/Os in flight
    __type(key, void *);        // The struct request * as a unique handle
    __type(value, __u64);        // Nanosecond start timestamp
} start_times SEC(".maps");

// Map 2: latency histogram — 64 linear buckets, each = 1ms
// Index 0 = [0, 1ms), Index 1 = [1ms, 2ms), Index 63 = [63ms, ∞)
struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, 64);
    __type(key, __u32);
    __type(value, __u64);
} latencies SEC(".maps");

// Map 3: separate read latency histogram
struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, 64);
    __type(key, __u32);
    __type(value, __u64);
} read_lat SEC(".maps");

// Map 4: separate write latency histogram
struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, 64);
    __type(key, __u32);
    __type(value, __u64);
}
```

```

} write_lat SEC(".maps");

// REQ_OP_WRITE is bit 0 of the request operation flags
#define REQ_OP_MASK 0xff

// kprobe fires at the ENTRY of blk_account_io_start()
// PT_REGS_PARM1 extracts the first argument: struct request *req
SEC("kprobe/blk_account_io_start")
int trace_io_start(struct pt_regs *ctx) {
    // Extract the struct request * from the first function argument
    void *req = (void *)PT_REGS_PARM1(ctx);

    // Record current monotonic nanosecond timestamp
    __u64 ts = bpf_ktime_get_ns();

    // Store timestamp keyed by the request pointer address
    // BPF_ANY = create or overwrite (safe if I/O overlaps)
    bpf_map_update_elem(&start_times, &req, &ts, BPF_ANY);
    return 0;
}

SEC("kprobe/blk_account_io_done")
int trace_io_done(struct pt_regs *ctx) {
    void *req = (void *)PT_REGS_PARM1(ctx);

    // Look up the start timestamp for this exact request
    __u64 *start_ts = bpf_map_lookup_elem(&start_times, &req);
    if (!start_ts) return 0; // Started before we loaded – skip

    __u64 delta_ns = bpf_ktime_get_ns() - *start_ts;

    // CRITICAL: delete the entry to prevent map bloat
    // If io_done fires without io_start, start_times grows unboundedly
    bpf_map_delete_elem(&start_times, &req);

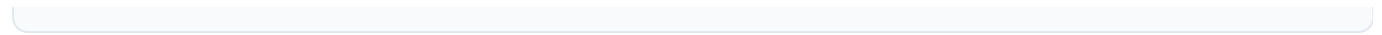
    // Convert ns → ms, clamp to bucket 63 for extreme outliers
    __u32 bucket = (__u32)(delta_ns / 1000000);
    if (bucket ≥ 64) bucket = 63;

    // Increment the combined histogram bucket (atomic – concurrent CPUs)
    __u64 *count = bpf_map_lookup_elem(&latencies, &bucket);
    if (count) __sync_fetch_and_add(count, 1);

    return 0;
}

char LICENSE[] SEC("license") = "GPL";

```



The Go Loader (`main.go`)

```
package main

import (
    "bufio"
    "fmt"
    "log"
    "os"
    "os/signal"
    "strings"
    "syscall"
    "time"

    "github.com/cilium/ebpf"
    "github.com/cilium/ebpf/link"
    "github.com/cilium/ebpf/rlimit"
)

func main() {
    log.Println("=====")
    log.Println("  eBPF Mastery Series: Lesson 03 – I/O Profiler  ")
    log.Println("=====")

    if err := rlimit.RemoveMemlock(); err != nil {
        log.Fatalf("Remove memlock: %v", err)
    }

    spec, err := ebpf.LoadCollectionSpec("main.bpf.o")
    if err != nil {
        log.Fatalf("Load spec: %v\nRun 'make' first.", err)
    }

    coll, err := ebpf.NewCollection(spec)
    if err != nil {
        log.Fatalf("Load collection: %v", err)
    }
    defer coll.Close()

    progStart := coll.Programs["trace_io_start"]
    progDone  := coll.Programs["trace_io_done"]
    latMap    := coll.Maps["latencies"]

    kpStart, err := link.Kprobe("blk_account_io_start", progStart, nil)
    if err != nil {
        log.Fatalf("kprobe attach (start): %v", err)
    }
}
```

```

}
defer kpStart.Close()

kpDone, err := link.Kprobe("blk_account_io_done", progDone, nil)
if err != nil {
    log.Fatalf("kprobe attach (done): %v", err)
}
defer kpDone.Close()

log.Println("Profiling block I/O latency. Histogram refreshes every 2s.")
log.Println("Commands:")
log.Println("  clear  - Reset all histogram counters")
log.Println("  show   - Print histogram immediately")
log.Println("  exit   - Detach and quit")
log.Println("_____")

ticker := time.NewTicker(2 * time.Second)
defer ticker.Stop()

sigChan := make(chan os.Signal, 1)
signal.Notify(sigChan, os.Interrupt, syscall.SIGTERM)

go func() {
    scanner := bufio.NewScanner(os.Stdin)
    for {
        fmt.Print("\n> ")
        if !scanner.Scan() { break }
        switch strings.TrimSpace(scanner.Text()) {
        case "clear":
            clearHistogram(latMap)
            fmt.Println("✓ Histogram counters reset.")
        case "show":
            printHistogram(latMap)
        case "exit", "quit":
            sigChan ← syscall.SIGTERM
        }
    }
}()

for {
    select {
    case ←ticker.C:
        printHistogram(latMap)
    case ←sigChan:
        fmt.Println("\nDetaching kprobes. Goodbye!")
        return
    }
}

```

```

    }
}

func printHistogram(m *ebpf.Map) {
    counts := make([]uint64, 64)
    var maxCount uint64
    for i := uint32(0); i < 64; i++ {
        _ = m.Lookup(&i, &counts[i])
        if counts[i] > maxCount { maxCount = counts[i] }
    }
    if maxCount == 0 {
        fmt.Println(" (no I/O recorded yet – try: dd if=/dev/urandom of=/tmp/x bs=1
        return
    }

    fmt.Printf("\n%-10s %-12s %s\n", "Latency", "Count", "Distribution")
    fmt.Println(strings.Repeat("-", 65))
    for i := uint32(0); i < 64; i++ {
        if counts[i] == 0 { continue }
        bar := strings.Repeat("█", int(counts[i]*40/maxCount))
        fmt.Printf("[%3d ms]   %-12d %s\n", i, counts[i], bar)
    }
}

func clearHistogram(m *ebpf.Map) {
    var zero uint64
    for i := uint32(0); i < 64; i++ {
        _ = m.Update(&i, &zero, ebpf.UpdateExist)
    }
}

```

10. Running the Lab

Phase 1: Compile & Run

```

cd series/series-03/source-code
make
sudo ./loader

```


Phase 2: Generate I/O Load in Another Terminal

```
# Write 100MB of random data to generate sustained I/O
dd if=/dev/urandom of=/tmp/test.bin bs=1M count=100

# Or run a database-like random-read workload
fio --name=rand-read --filename=/tmp/fio.bin --rw=randread \
    --bs=4k --size=500M --numjobs=4 --time_based --runtime=30
```

Phase 3: Observe the Live Histogram

Latency	Count	Distribution
[0 ms]	98432	
[1 ms]	4821	
[2 ms]	1023	
[5 ms]	142	
[50 ms]	3	← storage controller retry/timeout

Type `clear` to reset counters and see per-interval rates.

11. Debugging with bpftool

```
# Verify both kprobes are loaded and active
sudo bpftool prog list | grep trace_io

# Inspect the latency histogram map contents
sudo bpftool map list | grep latencies
sudo bpftool map dump id <ID>

# Check start_times map – should be near-empty (entries deleted on completion)
sudo bpftool map list | grep start_times
sudo bpftool map dump id <ID>
# If this shows thousands of entries, io_done kprobe may not be firing

# Find the actual function name on your kernel
sudo cat /proc/kallsyms | grep blk_account

# Stream debug output
sudo cat /sys/kernel/debug/tracing/trace_pipe
```

12. Verifier Errors & Common Mistakes

Error: kprobe attach fails with 'no such symbol'

```
failed to attach kprobe: no such function: blk_account_io_start
```

Cause: In kernel ≥ 5.16 , `blk_account_io_start` was renamed. Check:

```
sudo cat /proc/kallsyms | grep blk_account
# Use the exact name you find there
```

Error: `invalid BPF_XADD` or `unknown opcode d4`

Cause: `__sync_fetch_and_add()` on a map value requires the value pointer to be 8-byte aligned. Using `__u32` instead of `__u64` as the value type causes misalignment. **Fix:** Always use `__u64` for histogram bucket values.

Common Mistake: `start_times` map growing unboundedly

If `blk_account_io_done` never fires (hardware hang, incomplete I/O), entries accumulate in `start_times` until the map reaches `max_entries` and new entries start failing silently. **Fix:** In production, add a periodic user-space sweep that deletes entries older than 10 seconds.

13. Performance Analysis

Tool	Overhead	Granularity	Tail Latency	Production Safe
<code>iostat</code>	<0.1%	Device avg	No	Yes
<code>blktrace</code>	2–5%	Per-request	Yes	Sometimes
<code>strace</code>	100–1000x	Syscall	Yes	Never
eBPF Kprobe	<0.5%	Per-request	Yes	Yes

14. Best Practices

- **Use log2 buckets for μ s-range I/O:** Linear 1ms buckets lose all resolution for NVMe drives completing in 50–200 μ s. Log2 buckets (0–1 μ s, 1–2 μ s, 2–4 μ s...) cover 6 orders of magnitude with 64 buckets.
- **Separate read vs write histograms:** Reads and writes have fundamentally different latency profiles. Combining them masks write-amplification tail latency.
- **Clear counters after printing:** Snapshot the histogram, print it, reset to zero — gives you a per-interval view instead of cumulative totals that grow until they lose meaning.

15. Summary

- The Linux block layer represents every disk I/O as a `struct request` — the core unit for latency measurement.
 - Kprobes dynamically instrument kernel function entry points without reboot or kernel modification.
 - Using `struct request *` as the hash map key uniquely correlates async I/O start and completion events.
 - `bpf_ktime_get_ns()` provides monotonic, nanosecond-resolution timestamps immune to NTP drift.
 - Array-based histograms efficiently capture the full latency distribution without storing individual measurements.
-

16. Further Reading

- Brendan Gregg — BPF Performance Tools: bpf-performance-tools-book.html
- Datadog Disk Monitoring with eBPF: monitor-disk-performance
- Linux Block Layer Internals: [kernel.org block docs](https://kernel.org/doc/Documentation/block/README)
- Netflix FlameScope: github.com/Netflix/flamescope

Technical Deep-Dive Q&A: Block Device Latency Profiling with eBPF Kprobes

This guide contains detailed technical questions and model answers matching senior eBPF, Linux systems, storage engineering, and SRE roles.

1. Beginner Level

Q1. What is the difference between a Kprobe and a Kretprobe? Why do we need both for latency measurement?

Answer: Both probe types attach to kernel functions dynamically:

- **Kprobe** fires at the **entry** (first instruction) of a kernel function. It gives you access to the function's **input arguments** via `PT_REGS_PARM1(ctx)`, `PT_REGS_PARM2(ctx)`, etc.
- **Kretprobe** fires when the function **returns**. It gives you access to the **return value** via `PT_REGS_RC(ctx)`.

For latency measurement, you need **both** because a latency is the delta between two moments:

1. **T_start** — when `blk_account_io_start(req)` is called (use kprobe to capture this).
2. **T_end** — when `blk_account_io_done(req)` is called (use kprobe to capture this, then compute delta).

In our implementation, both are attached as kprobes (not kretprobes) because we care about when each function is *entered*, not what it returns. The `req` pointer — available as the first argument at function entry — is our correlation key across both events.

Q2. Why do we use `struct request *` as the BPF hash map key? Why not PID or thread ID?

Answer: Disk I/O is **asynchronous**. When a Postgres process calls `pread64()`, the kernel submits the I/O request to the block layer and the Postgres thread can return to doing other work (or sleep) while the hardware performs the transfer. The completion of that I/O may happen on a **different CPU core** and even handled by an interrupt handler — not the original Postgres thread.

If we used PID as the key:

- 16 concurrent I/Os from the same process would all share one entry and overwrite each other's timestamps.
- I/Os from kernel threads (like flusher daemons, `kworker`) have pseudo-PIDs that don't correspond to user processes.

The `struct request *` pointer is perfect because:

- It is a **unique memory address** in the kernel heap — no two in-flight I/Os share the same pointer.
- It is available as the first argument to both `blk_account_io_start` AND `blk_account_io_done`.
- Its lifetime matches exactly the I/O lifetime — allocated before `io_start`, freed after `io_done`.

2. Intermediate Level

Q3. Explain how `PT_REGS_PARM1(ctx)` extracts a function argument. What is `struct pt_regs` ?

Answer: When a kprobe fires, the kernel saves the CPU register state at the moment of the breakpoint into a `struct pt_regs`. This struct contains a snapshot of all CPU registers — `rax`, `rbx`, `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`, etc.

On x86_64, the **System V AMD64 ABI** (the Linux calling convention) specifies:

- **1st argument** → `rdi` register
- **2nd argument** → `rsi` register
- **3rd argument** → `rdx` register
- **4th argument** → `rcx` register

`PT_REGS_PARM1(ctx)` is a macro that expands to `ctx->di` — reading the `rdi` register from the saved state:

```
#define PT_REGS_PARM1(x) ((x)->di) // On x86_64
#define PT_REGS_PARM2(x) ((x)->si)
#define PT_REGS_PARM3(x) ((x)->dx)
```

So `void *req = (void *)PT_REGS_PARM1(ctx)` reads the value of the first argument to `blk_account_io_start()` — which is `struct request *req`.

On ARM64, the same macro reads `regs[0]`. The macro abstracts the architecture difference, making your kprobe code portable.

Q4. Why does our histogram use `__sync_fetch_and_add()` instead of a plain `count += 1`? What happens if you omit the atomic?

Answer: Multiple CPU cores run our kprobe simultaneously — each CPU core processes its own NVMe completion interrupt independently. If two cores simultaneously run `trace_io_done` and both try to increment `latencies[bucket]`:

Without atomic (plain `count += 1`):

1. CPU 0 reads `latencies[5] = 1000` → local register = 1000.
2. CPU 1 reads `latencies[5] = 1000` → local register = 1000.
3. CPU 0 writes 1001 back.
4. CPU 1 writes 1001 back. **CPU 0's increment is lost!**

This is a classic **lost update** race condition. Under heavy I/O load (NVMe at 1M IOPS), this could silently drop 10–20% of histogram counts.

`__sync_fetch_and_add(count, 1)` compiles to the `lock xadd` x86_64 instruction — an atomic read-modify-write that acquires the cache line exclusively before incrementing. No intermediate state is visible to other CPUs.

Performance note: For the absolute highest throughput, use `BPF_MAP_TYPE_PERCPU_ARRAY` (no atomics needed — each CPU has its own copy) and sum in user-space. For this lesson, the shared array with atomic is simpler and sufficient for typical disk I/O rates.

Q5. What happens if `blk_account_io_start` fires but `blk_account_io_done` never fires for a request? How does this cause a map leak?

Answer: Our `start_times` hash map has `max_entries = 10240`. For every entry inserted by `trace_io_start`, we expect `trace_io_done` to delete it after computing the delta.

If `io_done` never fires (hardware hang, I/O error causing the request to be aborted through a different code path, or kernel version change making our kprobe attach to the wrong function), entries accumulate:

1. `start_times` fills to 10240 entries.

2. New `bpf_map_update_elem(&start_times, ...)` calls return `-E2BIG`.
3. Since we use `BPF_ANY` flag, they actually succeed by overwriting the oldest entry — but this means the oldest entry's start time is **lost**.
4. When `io_done` eventually fires for that overwritten entry, the lookup returns a **newer** timestamp, producing a wildly incorrect (negative or near-zero) delta.

Production fix: Implement a TTL sweep. Store both the nanosecond timestamp AND the wall clock time. A user-space goroutine iterates the map every 30 seconds and deletes any entry older than 10 seconds — indicating a stuck or cancelled I/O.

3. Advanced Level

Q6. What is the fundamental instability risk of kprobes vs tracepoints? How would you detect if a kprobe silently stopped working after a kernel upgrade?

Answer: Kprobes attach by kernel **function name** looked up in `/proc/kallsyms`. The risk:

- **Inlining:** The compiler may inline `blk_account_io_start` into its callers, eliminating it as a standalone function. Your kprobe attach returns success (the symbol still exists in kallsyms as a dead label), but the probe never fires.
- **Renaming:** In kernel 5.16, `blk_account_io_start` was renamed. Any eBPF tool using the old name returns "no such symbol" and fails — **visibly**.
- **Signature change:** The argument order or type of an argument changes. Your kprobe still attaches, but `PT_REGS_PARM1(ctx)` now extracts the wrong argument. Data is silently corrupted.

Detection strategy:

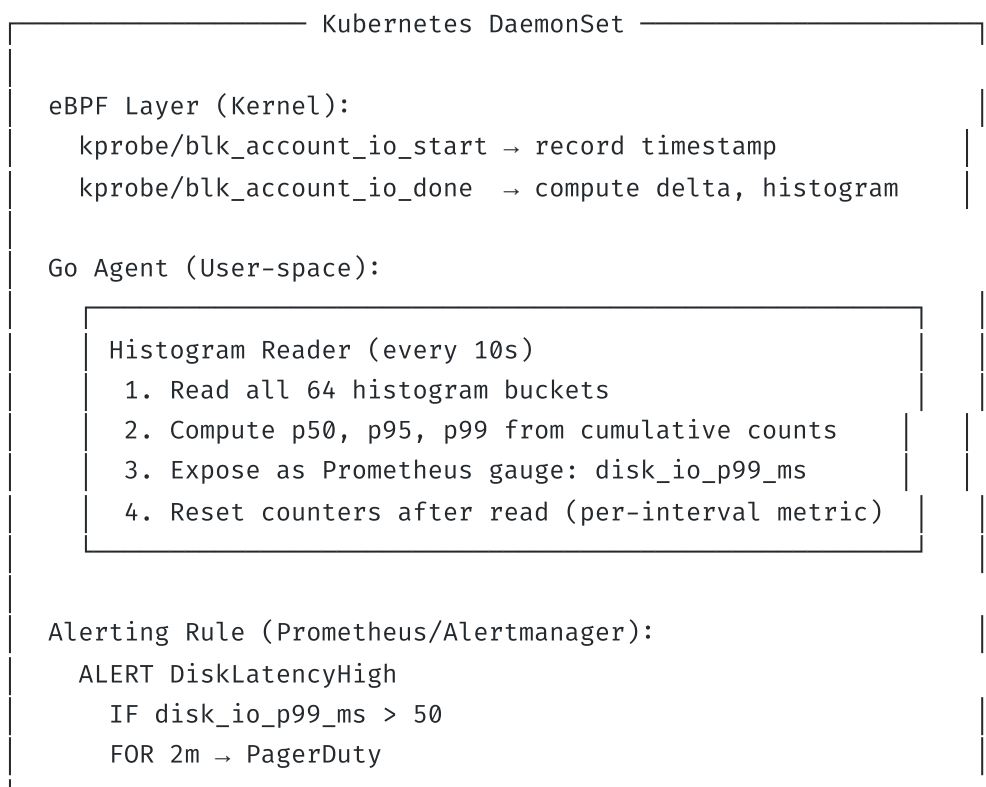
1. **Self-test counter:** Add a `BPF_MAP_TYPE_PERCPU_ARRAY` counter that `trace_io_start` increments on every call. In user-space, verify this counter is increasing when you know I/O is happening (write a test file). If the counter stays zero, the kprobe isn't firing.
 2. **Correlation check:** If `trace_io_done` fires 1000 times but `trace_io_start` only fired 100 times (check separate counters), your start kprobe is broken.
 3. **Fallback to tracepoints:** Check `sudo ls /sys/kernel/debug/tracing/events/block/` — the block subsystem has stable tracepoints like `block_rq_issue` and `block_rq_complete` that are equivalent and stable.
-

4. System Design & Scenario-Based

Q7. Design a production SRE tool that automatically detects disk I/O tail latency regressions and pages an on-call engineer.

Answer:

Architecture:



Computing p99 from histogram buckets:

```
func computePercentile(counts []uint64, pct float64) int {
    var total uint64
    for _, c := range counts { total += c }
    threshold := uint64(float64(total) * pct)
    var cumulative uint64
    for i, c := range counts {
        cumulative += c
        if cumulative ≥ threshold { return i }
    }
    return 63
}

p99 := computePercentile(counts, 0.99) // Returns ms bucket index
```

Key design decisions:

1. **Per-device histograms:** Check `req→q→disk→disk_name` (with CO-RE) to build separate histograms per device (`sda` , `nvme0n1`), enabling pinpointing which disk is slow.
2. **Per-process histograms:** Add a map keyed by `{device, pid}` for per-database-instance visibility.

5. Troubleshooting & Debugging

Q8. You deployed the latency profiler but the histogram always shows 0 counts, even while `dd` is writing 1GB of data. What are the 4 most likely causes?

Answer:

1. **Function name mismatch:** `blk_account_io_start` may not exist on your kernel. Verify:

```
sudo cat /proc/kallsyms | grep -i "blk_account"
```

If empty or different name, update the SEC() annotation in your C code.

2. **kprobe silently not firing due to inlining:** The function exists in kallsyms but has been inlined by the compiler. Verify the kprobe is actually receiving events:

```
sudo bpftool prog list # Find your prog ID
sudo bpftool prog show id <ID> # Check run_cnt - should increase
```

3. **dd uses O_DIRECT but kprobe is at wrong layer:** With `O_DIRECT`, writes bypass the page cache and go directly to the block device. Without `O_DIRECT`, small writes may be buffered in the page cache and never reach `blk_account_io_start` during your test window. Use `oflag=direct` flag in dd:

```
dd if=/dev/zero of=/tmp/test.bin bs=1M count=100 oflag=direct
```

4. **Writing to a tmpfs or ramfs filesystem:** `/tmp` on many modern Linux distributions is mounted as `tmpfs` (RAM-based filesystem). There are no block device operations for `tmpfs` writes — they go directly to RAM, never touching the block layer. Check with `df -T /tmp`. Use a path on a real block device instead (e.g., `/var/tmp` or a mounted disk).

Hands-on Lab & Homework Assignments

1. Practical Exercises & Research Tasks

Task 1: Explore the Block Layer Tracepoints

Before using kprobes, always check if a stable tracepoint exists:

```
# List all block-layer tracepoints
sudo ls /sys/kernel/debug/tracing/events/block/

# Read the argument format for block_rq_issue (request issued to hardware)
sudo cat /sys/kernel/debug/tracing/events/block/block_rq_issue/format

# Read the format for block_rq_complete
sudo cat /sys/kernel/debug/tracing/events/block/block_rq_complete/format
```

- What fields are available in `block_rq_complete` that our kprobe can't easily read? (Hint: look for `nr_sector` and `rwbs`.)
- Why would `tracepoint/block/block_rq_issue` + `tracepoint/block/block_rq_complete` be a more production-stable alternative to our kprobe approach?

Task 2: Inspect Kprobes with bpftool

After running `sudo ./loader`, in a separate terminal:

```
# List all loaded eBPF programs
sudo bpftool prog list

# Find trace_io_start and trace_io_done — note their IDs
sudo bpftool prog show name trace_io_start
sudo bpftool prog show name trace_io_done

# Check how many times each has fired (run_cnt)
sudo bpftool prog show id <ID> --pretty

# Dump the JIT-compiled machine code
sudo bpftool prog dump jited name trace_io_start

# Inspect the start_times map — should have very few entries (near-empty)
sudo bpftool map list | grep start_times
sudo bpftool map dump id <ID>
```

- If `run_cnt` for `trace_io_start` is 0 after 30 seconds of disk I/O, what is the problem?
- What does a non-empty `start_times` map (with many entries) indicate about your I/O workload?

2. Coding Assignment: Read vs Write Latency Separation

Objective: Split the single combined histogram into two separate histograms — one for reads and one for writes.

Requirements:

1. Declare two additional histogram maps in `main.bpf.c` :

```

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, 64);
    __type(key, __u32);
    __type(value, __u64);
} read_lat SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, 64);
    __type(key, __u32);
    __type(value, __u64);
} write_lat SEC(".maps");

```

2. In `trace_io_done`, also pass the `req` pointer to detect read vs write. Store the operation type in the `start_times` map value — change the value from `__u64` to a struct:

```

struct io_event {
    __u64 start_ns;
    __u8 is_write; // 1 = write, 0 = read
};

```

Read the operation from the request in `trace_io_start`:

```

// PT_REGS_PARM2 gives access to the second argument
// For blk_account_io_start(struct request *req), check req->cmd_flags
// Use bpf_probe_read_kernel to safely dereference req->cmd_flags

```

3. In `trace_io_done`, read `io_event.is_write` and increment either `read_lat[bucket]` or `write_lat[bucket]`.
4. Update `main.go` to:
- Add separate `read_lat` and `write_lat` map handles.
 - Print two histograms side-by-side: "READ LATENCY" and "WRITE LATENCY".
 - Add a CLI command `mode read|write|both` to toggle which histogram is displayed.
5. Verify: While running `sudo ./loader`, test with:

```
# Pure reads – should appear in read histogram only
dd if=/dev/sda of=/dev/null bs=1M count=100

# Pure writes – should appear in write histogram only
dd if=/dev/zero of=/tmp/write_test.bin bs=1M count=100 oflag=direct
```

3. Mini-Project: Log2 Histogram (μ s Resolution)

Objective: Implement a logarithmic-scale histogram for NVMe drives that complete I/O in 50–500 microseconds (where 1ms linear buckets are too coarse).

Design:

Replace the linear 1ms buckets with log2 buckets:

- Bucket 0 = [0, 1 μ s)
- Bucket 1 = [1 μ s, 2 μ s)
- Bucket 2 = [2 μ s, 4 μ s)
- Bucket 3 = [4 μ s, 8 μ s)
- ...
- Bucket 30 = [536ms, 1073ms)

```
// In trace_io_done:
__u64 delta_ns = bpf_ktime_get_ns() - *start_ts;
__u32 bucket = 0;
__u64 val = delta_ns / 1000; // Convert to microseconds

// Compute floor(log2(val)) using bit-scan reverse
if (val > 0) {
    bucket = 64 - __builtin_clzll(val) - 1;
}
if (bucket ≥ 64) bucket = 63;
```

In the Go printer, display the bucket ranges with proper unit labels:

Latency Range	Count	Distribution
[0–1 μ s]	0	
[1–2 μ s]	4210	
[2–4 μ s]	98432	
[4–8 μ s]	8921	
[512ms+]	3	← NVMe timeout retries

4. Advanced Challenge: Per-Process I/O Profiler

Objective: Build a composite key `{PID, device_major}` to show per-process disk latency histograms, like Datadog's NPM.

Design:

```
struct proc_dev_key {
    __u32 pid;
    __u32 dev; // major << 20 | minor from req→q
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 1024);
    __type(key, struct proc_dev_key);
    __type(value, __u64); // store in a per-key histogram
} per_proc_lat SEC(".maps");
```

In user-space, display a table like:

PID	COMM	DEVICE	p50	p95	p99	
9821	postgres	sda	0ms	2ms	48ms	← PROBLEM
1234	nginx	sda	0ms	0ms	1ms	
8901	kworker	sda	0ms	1ms	3ms	

Challenge: The `p99` for Postgres is 48ms while nginx on the same device is 1ms. This proves the issue is not the device itself (that would affect all processes equally) — it is I/O isolation. A Kubernetes `LimitRange` storage QoS limit or a `blkio` cgroup throttle is needed.

Write a short analysis (200 words) explaining what this data reveals about the Kubernetes storage QoS policy.